

Deque (Double-Ended Queue)

Input-Restricted and Output-Restricted Deques

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

What is a Deque?

Definition

A **Deque** (Double-Ended Queue) is a linear data structure in which elements can be inserted or deleted from either end.

The two ends are usually called:

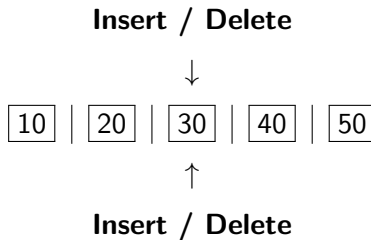
LEFT / FRONT

RIGHT / REAR

A general deque allows:

- insertion at the left,
- insertion at the right,
- deletion from the left,
- deletion from the right.

Visualizing a Deque



Operations can take place at both ends

Queue vs Deque

Property	Queue	Deque
Insertion	Rear	Both ends
Deletion	Front	Both ends
Behaviour	FIFO	Flexible
Pointers	Front, Rear	Left, Right

Important

Insertion and deletion are not performed from arbitrary positions in the middle of a deque.

Representation of a Deque

A deque can be implemented using:

- 1 Circular Array
- 2 Circular Doubly Linked List

For an array implementation:

```
int deque[MAX];
```

Two variables are maintained:

left *right*

Initially:

$left = right = -1$

This represents an empty deque.

Why Circular Array?

Consider an array of size 5.

10	20	30	40	50
0	1	2	3	4

Suppose elements at positions 0 and 1 are deleted.

A linear implementation may waste these locations.

A circular implementation allows:

$$4 \rightarrow 0$$

Therefore, unused locations can be reused.

Circular Movement

Moving RIGHT forward:

$$right = (right + 1) \bmod MAX$$

Moving LEFT forward:

$$left = (left + 1) \bmod MAX$$

Moving LEFT backward:

$$left = (left - 1 + MAX) \bmod MAX$$

Moving RIGHT backward:

$$right = (right - 1 + MAX) \bmod MAX$$

Types of Deques

Two restricted forms of deque are commonly studied:

① Input-Restricted Deque

Insertion is allowed at only one end, but deletion is allowed from both ends.

② Output-Restricted Deque

Deletion is allowed at only one end, but insertion is allowed at both ends.

Input-Restricted Deque

Suppose insertion is restricted to the RIGHT end.

Allowed:

- Insert Right
- Delete Left
- Delete Right

Not allowed:

Insert Left

Therefore:

One insertion end + Two deletion ends

Output-Restricted Deque

Suppose deletion is restricted to the LEFT end.

Allowed:

- Insert Left
- Insert Right
- Delete Left

Not allowed:

Delete Right

Therefore:

Two insertion ends + One deletion end

Input vs Output Restricted Deque

Operation	Input Restricted	Output Restricted
Insert Left	No	Yes
Insert Right	Yes	Yes
Delete Left	Yes	Yes
Delete Right	Yes	No

Memory Trick

Input restricted $\Rightarrow$ restrict where data enters.

Output restricted $\Rightarrow$ restrict where data leaves.

Empty Condition

Initially:

$$left = -1$$

and

$$right = -1$$

Therefore:

$$left = -1$$

can be used to identify an empty deque.

When the last remaining element is deleted:

$$left = right$$

we reset:

Full Condition

For a circular deque, the structure is full when the next position of RIGHT is LEFT.

$$(right + 1) \bmod MAX = left$$

Equivalent traditional conditions are:

$$left = 0 \quad \text{and} \quad right = MAX - 1$$

or

$$left = right + 1$$

The modulo expression is easier to implement.

Insertion at Right

Algorithm:

- 1 Check whether the deque is full.
- 2 If empty:

$$left = right = 0$$

- 3 Otherwise:

$$right = (right + 1) \bmod MAX$$

- 4 Insert the value:

$$deque[right] = value$$

Complexity:

$$O(1)$$

Insert at Right – C Code

```
1 int insertRight(int value)
2 {
3     if ((right + 1) % MAX == left)
4     {
5         printf("OVERFLOW\n");
6         return 0;
7     }
8
9     if (left == -1)
10    {
11        left = 0;
12        right = 0;
13    }
14    else
15    {
16        right = (right + 1) % MAX;
17    }
18
19    deque[right] = value;
20
21    return 1;
22 }
```

Insertion at Left

Algorithm:

- 1 Check whether the deque is full.
- 2 If empty:

$$left = right = 0$$

- 3 Otherwise:

$$left = (left - 1 + MAX) \bmod MAX$$

- 4 Store:

$$deque[left] = value$$

Complexity:

$$O(1)$$

Insert at Left – C Code

```
1 int insertLeft(int value)
2 {
3     if ((right + 1) % MAX == left)
4     {
5         printf("OVERFLOW\n");
6         return 0;
7     }
8
9     if (left == -1)
10    {
11        left = 0;
12        right = 0;
13    }
14    else
15    {
16        left = (left - 1 + MAX) % MAX;
17    }
18
19    deque[left] = value;
20
21    return 1;
22 }
```

Deletion from Left

Algorithm:

① Check underflow.

② Retrieve:

$$value = deque[left]$$

③ If:

$$left = right$$

then:

$$left = right = -1$$

④ Otherwise:

$$left = (left + 1) \bmod MAX$$

Complexity:

$$O(1)$$

Delete from Left – C Code

```
1 int deleteLeft(int *value)
2 {
3     if (left == -1)
4     {
5         printf("UNDERFLOW\n");
6         return 0;
7     }
8
9     *value = deque[left];
10
11    if (left == right)
12    {
13        left = -1;
14        right = -1;
15    }
16    else
17    {
18        left = (left + 1) % MAX;
19    }
20
21    return 1;
22 }
```

Deletion from Right

Algorithm:

- 1 Check underflow.
- 2 Retrieve:

$$value = deque[right]$$

- 3 If:

$$left = right$$

reset both to -1 .

- 4 Otherwise:

$$right = (right - 1 + MAX) \bmod MAX$$

Complexity:

$$O(1)$$

Delete from Right – C Code

```
1 int deleteRight(int *value)
2 {
3     if (left == -1)
4     {
5         printf("UNDERFLOW\n");
6         return 0;
7     }
8
9     *value = deque[right];
10
11    if (left == right)
12    {
13        left = -1;
14        right = -1;
15    }
16    else
17    {
18        right = (right - 1 + MAX) % MAX;
19    }
20
21    return 1;
22 }
```

Let's Trace a Deque

Assume:

$$MAX = 5$$

Initially:

$$left = right = -1$$

Perform:

- ① InsertRight(10)
- ② InsertRight(20)
- ③ InsertLeft(5)
- ④ InsertRight(30)
- ⑤ DeleteLeft()
- ⑥ InsertRight(40)

Try to calculate LEFT and RIGHT after every operation.

Dry Run Solution

Step	Operation	Left	Right	Logical Deque
0	Initial	-1	-1	Empty
1	InsertRight(10)	0	0	10
2	InsertRight(20)	0	1	10,20
3	InsertLeft(5)	4	1	5,10,20
4	InsertRight(30)	4	2	5,10,20,30
5	DeleteLeft()	0	2	10,20,30
6	InsertRight(40)	0	3	10,20,30,40

Physical vs Logical Order

After inserting 5 at the left:

$$left = 4, \quad right = 1$$

Physical array:

10	20	-	-	5
0	1	2	3	4

But the logical deque is:

5, 10, 20

Key Idea

Physical array order and logical deque order can be different.

Basic C Definitions

```
1 #include <stdio.h>
2
3 #define MAX 10
4
5 int deque[MAX];
6
7 int left = -1;
8 int right = -1;
9
10 int isEmpty(void)
11 {
12     return (left == -1);
13 }
14
15 int isFull(void)
16 {
17     return ((right + 1) % MAX == left);
18 }
```

Display Function

```
1 void display(void)
2 {
3     int i;
4
5     if (left == -1)
6     {
7         printf("DEQUE IS EMPTY\n");
8         return;
9     }
10
11     i = left;
12
13     printf("LEFT -> ");
14
15     while (1)
16     {
17         printf("%d ", deque[i]);
18
19         if (i == right)
20             break;
21
22         i = (i + 1) % MAX;
23     }
24
25     printf("<- RIGHT\n");
26 }
```

Complexity Analysis

Operation	Time Complexity
Insert Left	$O(1)$
Insert Right	$O(1)$
Delete Left	$O(1)$
Delete Right	$O(1)$
Peek Left	$O(1)$
Peek Right	$O(1)$
Display	$O(n)$

For an array of capacity MAX :

$$\text{Space Complexity} = O(MAX)$$

Can a Deque Behave Like a Stack?

Yes.

Use:

InsertRight()

for PUSH and:

DeleteRight()

for POP.

Therefore:

LIFO

A deque can implement stack behaviour without a separate stack data structure.

Can a Deque Behave Like a Queue?

Yes.

Use:

InsertRight()

for ENQUEUE and:

DeleteLeft()

for DEQUEUE.

Therefore:

FIFO

Hence, a deque is more flexible than a conventional queue.

Applications of Deques

Dequeues are useful in:

- Palindrome checking
- Sliding-window problems
- Undo/Redo systems
- Task scheduling
- Browser navigation
- Work-stealing algorithms
- Graph algorithms
- 0-1 BFS
- Cache-related algorithms

Example: Palindrome Checking

Consider:

LEVEL

Insert all characters into a deque.

Then repeatedly compare:

DeleteLeft()

with:

DeleteRight()

For LEVEL:

$L = L$

$E = E$

Quick Check

- 1 What does DEQUE stand for?
- 2 Which operation is restricted in an input-restricted deque?
- 3 Which operation is restricted in an output-restricted deque?
- 4 What is the empty condition?
- 5 What is the full condition?

Think Before Answering

Suppose:

$$MAX = 5$$

and:

$$left = 2, \quad right = 1$$

Is the deque full?

Use:

$$(right + 1) \bmod MAX$$

Think Before Answering

Suppose:

$$MAX = 5$$

and:

$$left = 2, \quad right = 1$$

Is the deque full?

Use:

$$(right + 1) \bmod MAX$$

$$(1 + 1) \bmod 5 = 2$$

Since:

Trace This

Initially the deque is empty.

Perform:

- ① InsertRight(10)
- ② InsertRight(20)
- ③ InsertLeft(5)
- ④ DeleteRight()
- ⑤ InsertLeft(2)

Questions:

- Which element is deleted?
- What is the final logical deque?

Trace This

Initially the deque is empty.

Perform:

- 1 InsertRight(10)
- 2 InsertRight(20)
- 3 InsertLeft(5)
- 4 DeleteRight()
- 5 InsertLeft(2)

Questions:

- Which element is deleted?
- What is the final logical deque?

Deleted element:

20

Final deque:

Find the Mistake

A student writes:

$$\textit{right} = \textit{MAX} - 1$$

as the full condition for a deque.
Is this always correct?

Find the Mistake

A student writes:

$$right = MAX - 1$$

as the full condition for a deque.
Is this always correct?

No.

Because empty positions may exist at the beginning of the circular array.
Correct condition:

$$(right + 1) \bmod MAX = left$$

Coding Challenge

Modify the deque implementation to add:

- 1 `peekLeft()`
- 2 `peekRight()`
- 3 `size()`
- 4 `search()`
- 5 `clear()`

Challenge

Can you implement `size()` in $O(1)$ time?

Summary

- Deque means Double-Ended Queue.
- Operations can occur at both ends.
- Input-restricted deque restricts insertion.
- Output-restricted deque restricts deletion.
- Circular arrays prevent unnecessary wastage of space.
- Empty condition:

$$left = -1$$

- Full condition:

$$(right + 1) \bmod MAX = left$$

- Insertion and deletion at either end take $O(1)$ time.

Before leaving, answer:

- ① Why is a deque more flexible than a queue?
- ② Differentiate input-restricted and output-restricted deque.
- ③ Why is a circular array preferred?
- ④ Write the overflow condition.
- ⑤ How can a deque simulate a stack?
- ⑥ How can a deque simulate a queue?

References

- ① Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
- ② Reema Thareja, *Data Structures Using C*, Oxford University Press.
- ③ A. M. Tenenbaum, *Data Structures Using C and C++*.

Your Feedback Matters

Please share your feedback about:

- Understanding of Deques
- Input-Restricted Deque
- Output-Restricted Deque
- Circular representation
- Coding examples
- Pace of the lecture
- Suggestions for improvement

Thank you for your feedback!



Scan to Submit Feedback

Thank You

Questions?